

Conditional DETR

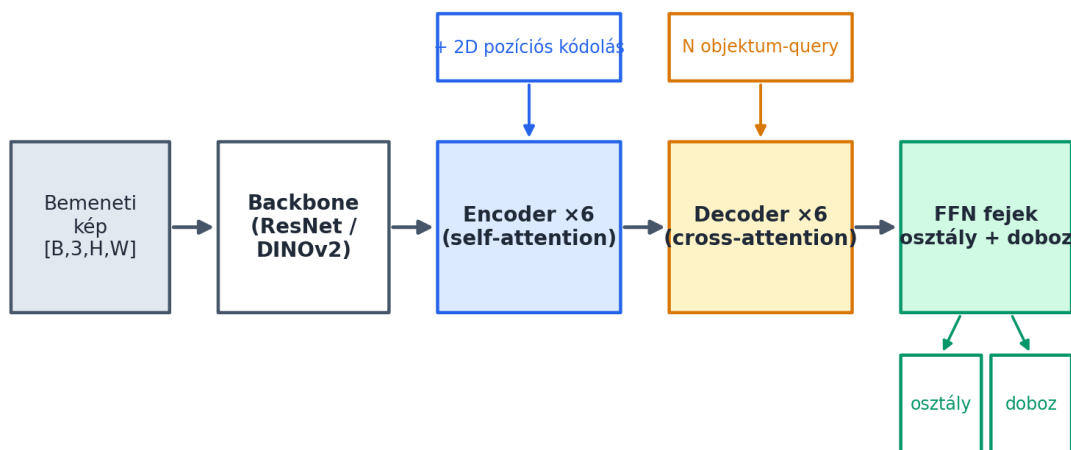
A feltételes térbeli lekérdezés

Hogyan tanul nagyságrenddel gyorsabban egy Detection Transformer, ha a „**mit?**” és a „**hol?**” kérdést szétválasztjuk a dekóder figyelmi mechanizmusában — olvasmányosan, de a szükséges technikai részletekkel.

1. Mit csinál a DETR?

A DETR (*DEtection TRansformer*, Carion és társai, 2020) újragondolta az objektumdetektálást. A klasszikus detektorokkal ellentétben nincs benne előre kijelölt horgony (*anchor*), és nincs utólagos átfedés-szűrés (*non-maximum suppression*) sem. Helyette a feladatot **halmazpredikcióként** fogalmazza meg: a modell egyszerre, közvetlenül ad vissza egy rögzített méretű halmaznyi objektumot.

A működés négy lépésben foglalható össze. Egy **backbone** (például ResNet vagy DINOv2) jellemzőtérképet készít a képből; ehhez 2D szinuszos **pozíciós kódolást** adunk, hogy a Transformer tudja, melyik jellemző hol helyezkedik el. Az **encoder** önfigyelemmel (self-attention) minden képpontot összekapcsol minden másikkal. A **dekóder** N darab tanulható „objektum-query”-t kap; mindegyik query cross-attentionnel keresi meg a saját objektumát az encoder kimenetén. Végül két kis fejhálózat (FFN) minden query-ből egy **osztályt** és egy **bounding boxot** jósol. Tanításkor egy páros illesztés (a magyar matematikusról elnevezett *Hungarian matching*) rendeli hozzá a jóslatokat a valódi objektumokhoz.



1. ábra — A DETR feldolgozási lánc: backbone → encoder → dekóder → predikciós fejek.

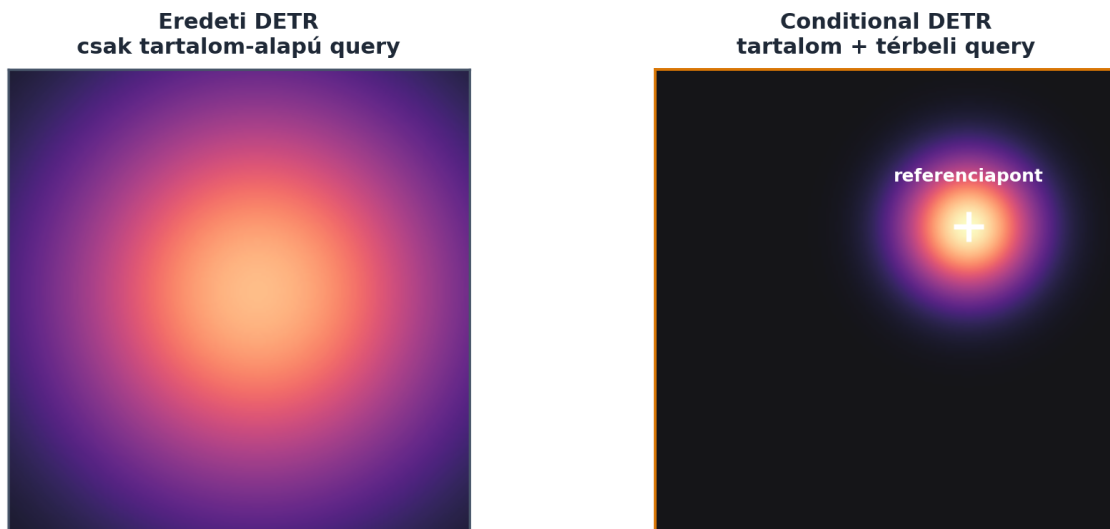
A lényeg dióhéjban

A DETR eleganciájának ára a **lassú tanulás**: az eredeti modell nagyjából 500 epochot igényel. A Conditional DETR pontosan ezt a gyenge pontot orvosolja — egyetlen, jól irányzott változtatással a dekóder figyelmében.

2. Miért lassú az eredeti DETR?

A lassúság a dekóder **cross-attentionjében** gyökerezik. Itt minden query azt próbálja megtalálni, hogy a képen hol vannak az adott objektum lényeges pontjai — jellemzően a doboz szélei. A gond az, hogy a query térbeli „iránytűje” egyetlen, minden képre **közös**, tanult vektor (a query pozíciós beágyazása). Ez önmagában nem tudja megmondani, hol van az objektum egy konkrét képen; ehhez a query-nek a saját **tartalmi** embeddingjére kellene támaszkodnia.

Csak hogy a tartalmi embedding a tanítás elején még lényegében „üres” — nem hordoz megbízható helyzetinformációt. Emiatt a figyelem sokáig szétterül az egész képen (2. ábra, bal oldal), és nagyon sok epoch kell, mire a query-k egyáltalán megtanulják, hova érdemes nézniük. A modell gyakorlatilag a **nulláról** tanulja meg a térbeli lokalizációt, ami rendkívül lassú folyamat.



a figyelem szétterül → lassú tanulás

a figyelem a referencia köré fókuszál → gyors

2. ábra — Bal: tartalom-alapú query, szétterülő figyelem (lassú). Jobb: tartalom + térbeli query, a referenciapont köré fókuszáló figyelem (gyors).

3. Az ötlet: válaszd szét a „mit?” és a „hol?” kérdést

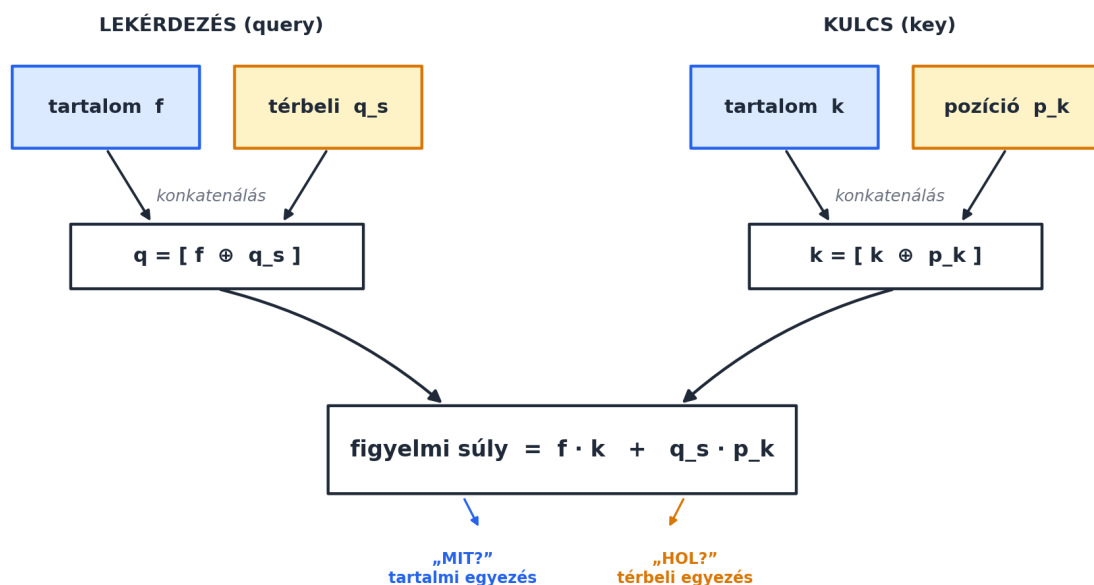
A Conditional DETR (Meng és társai, 2021) kulcsgondolata, hogy a **lokalizációt leveszi a tartalmi embedding válláról**, és külön, kifejezett térbeli jelként adja a query-hez. Ehhez három lépésben építi fel a feltételes térbeli lekérdezést.

1) Referenciapont. Minden query-hez rendel egy referenciapontot, s -t — ez egy 2D koordináta a képen, amelyet a query pozíciós beágyazásából egy tanult lineáris réteg és egy sigmoid állít elő.

2) Szinuszos beágyazás. A referenciapontból szinuszos beágyazást képez, p_s -t. Fontos, hogy ez ugyanabban a „nyelvben” (ugyanazzal a szinuszos sémával) íródik, mint a kép pozíciós kódolása — erre a 4. szakaszban még visszatérünk.

3) Feltételes térbeli query. A dekóder tartalmi kimenetéből egy $T = FFN(f)$ transzformációt tanul, és a térbeli query: $q_s = T \odot p_s$ (elemenkénti szorzat). A T teszi lehetővé, hogy minden query a saját tartalmához igazítsa a térbeli keresést.

A cross-attentionben ezután a lekérdezés és a kulcs is **két rész összefűzése** (konkatenációja): a query a tartalomból (f) és a térbeli tagból (q_s), a kulcs pedig a memória tartalmából (k) és annak pozíciós kódolásából (p_k) áll. Így a figyelmi súly két, egymástól független tagra bomlik (3. ábra).



3. ábra — A dekuplázott figyelmi súly: a tartalmi és a térbeli egyezés külön tagként adódik össze.

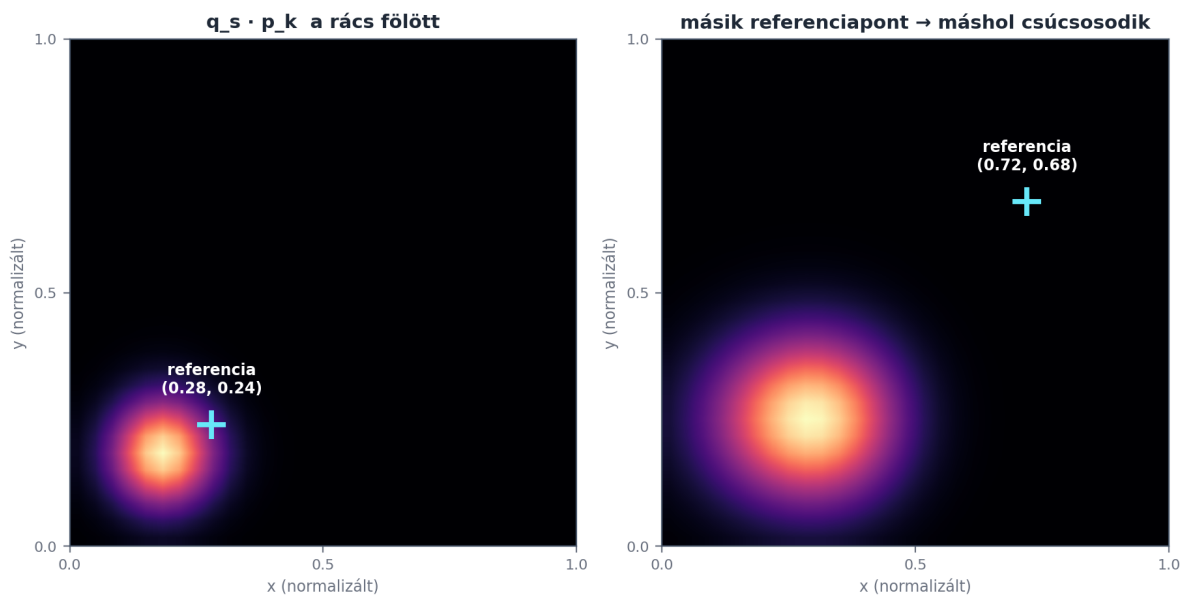
A dekuplázás a lényeg

Az első tag a **„MIT?”** kérdésre felel (a megjelenés egyezése), a második a **„HOL?”** kérdésre (a térbeli pozíció egyezése). A régi DETR ezt a két hatást összeadta egyetlen query-be, így összemosódtak; a Conditional DETR **szétválasztja** őket, hogy külön-külön, tisztán dolgozhassanak.

4. Miért éppen a referenciapont köré fókuszál a figyelem?

A második tag, $q_s \cdot p_k$, azért működik, mert p_s (a referenciapont beágyazása) és p_k (a rács pontjainak beágyazása) **ugyanabban a szinuszos térben** él. Két hely szinuszos beágyazásának skaláris szorzata annál nagyobb, minél közelebb van egymáshoz a két hely. Ezért amikor a referenciapont beágyazását összevetjük a jellemzőtérkép minden pontjának pozíciós beágyazásával, a szorzat éppen a **referenciapont környékén** lesz a legnagyobb.

A 4. ábra ezt **valós számítással** mutatja: egy-egy referenciapont beágyazása és egy 30×30 -as rács pozíciós beágyazásai közötti skaláris szorzatot ábrázoltuk. A figyelem tisztán a referenciapont köré összpontosul, és ha a referenciapont máshova kerül, a fókusz is vele mozog. Ezért a query már a tanítás legelső lépéseitől a „helyes környéken” keres, ahelyett hogy vakon tapogatózna — ez adja a nagyságrendi gyorsulást.



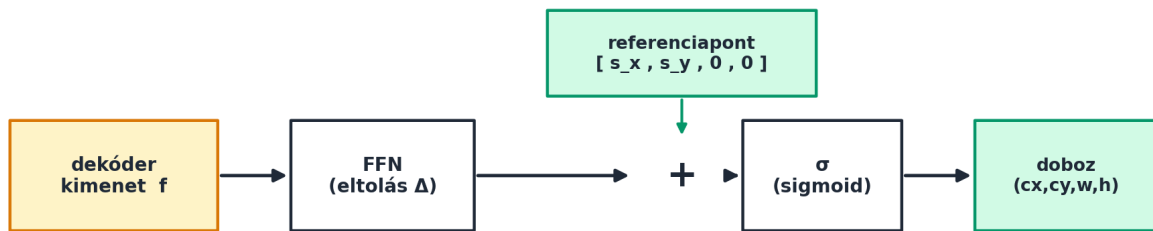
4. ábra — Két különböző referenciapontra kiszámolt térbeli figyelem. A világos folt mindig a referenciapont (kék +) köré esik.

5. A dobozfej: eltolás a referenciaponthoz képest

A referenciapont a bounding box jóslásában is szerepet kap. Ahelyett, hogy a hálónak a doboz abszolút helyét kellene a semmiből megtippelnie, csak a referenciaponthoz képesti **eltolást** kell megtanulnia:

$$b = \sigma(\text{FFN}(f) + [s_x, s_y, 0, 0])$$

A referencia (x, y) koordinátáit még a sigmoid (σ) előtt hozzáadjuk a jósolt eltoláshoz, a szélesség- és magasságtagokhoz pedig nullát. Ez a „jó kiindulópont” tovább könnyíti és gyorsítja a tanulást: a modellnek finomhangolnia kell, nem nulláról felépítenie a doboz helyét.



$$b = \sigma(\text{FFN}(f) + [s_x, s_y, 0, 0]) \rightarrow \text{csak az eltolást kell tanulni}$$

5. ábra — A dobozfej a referenciapontot használja kiindulásként, és csak az ahhoz képesti eltolást tanulja.

6. Összefoglalás

A Conditional DETR egyetlen, jól irányzott változtatással — a **térbeli és a tartalmi keresés szétválasztásával** — drasztikusan felgyorsítja a tanulást, miközben megtartja a DETR eleganciáját: horgony és utósűrítés nélküli, végponttól végpontig tanuló detektor marad. A szerzők szerint a modell nagyjából **6,7-szer** gyorsabban konvergál a ResNet-50/101 backbone-okkal, és akár **10-szer** gyorsabban az erősebb DC5 változatokkal — az eredeti ~500 epochnak így már a töredéke is elég, hasonló pontosság mellett.

Szempont	Eredeti DETR	Conditional DETR
Cross-attention query	csak tartalom (+ közös query-pozíció)	tartalom $\oplus$ feltételes térbeli (q_s)
Térbeli lokalizáció	a tartalmi embeddingből, közvetetten	kifejezett referenciapont + szinuszos beágyazás
Figyelmi súly	összemosott	szétválik: $f \cdot k + q_s \cdot p_k$
Dobozjóslás	abszolút hely	eltolás a referenciaponthoz képest
Konvergencia	~500 epoch	~6,7–10× gyorsabb

Kapcsolat a kóddal

A csatolt `detr.py`-ban a `ConditionalDecoderLayer` és a `ConditionalCrossAttention` pontosan ezt a mechanizmust valósítja meg: a megosztott referenciapontokat, a $T \odot p_s$ térbeli query-t és a konkatenált, dekoplázott cross-attentiont.